

# Payment Transaction Management System

## Database Management System – Week 3 Task

|                |                 |
|----------------|-----------------|
| Name           | Akash L         |
| Enrollment No. | TU6253210211001 |
| Week           | 3               |

### Description

- Create a Payment table to store transaction records.
- Store payment details such as mode, date, and status.
- Manage successful and failed transactions.
- Analyze payment methods used by customers.
- Generate payment transaction reports.

### 1. Creating the Payment Table

The Payment table stores each transaction along with the customer/order it belongs to, the mode of payment, the date of payment, the amount, and its status.

```
CREATE TABLE Payment (  
  payment_id      INT PRIMARY KEY AUTO_INCREMENT,  
  order_id        INT NOT NULL,  
  customer_id     INT NOT NULL,  
  payment_mode    VARCHAR(20) NOT NULL,    -- e.g. Cash, Card, UPI, Net Banking  
  payment_date    DATE NOT NULL,  
  amount          DECIMAL(10,2) NOT NULL,  
  status          VARCHAR(15) NOT NULL,    -- e.g. Success, Failed, Pending  
  FOREIGN KEY (order_id) REFERENCES Orders(order_id),  
  FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)  
);
```

### 2. Storing Payment Details (Mode, Date, Status)

```
INSERT INTO Payment (order_id, customer_id, payment_mode, payment_date, amount, status)  
VALUES  
  (101, 1, 'UPI',          '2026-08-01', 2500.00, 'Success'),  
  (102, 2, 'Credit Card', '2026-08-02', 1500.00, 'Success'),  
  (103, 3, 'Net Banking',  '2026-08-03', 3200.00, 'Failed'),  
  (104, 1, 'Cash',         '2026-08-04',  800.00, 'Success'),  
  (105, 4, 'UPI',          '2026-08-05', 1200.00, 'Failed'),  
  (106, 2, 'Debit Card',   '2026-08-06', 4500.00, 'Success');
```

### 3. Managing Successful and Failed Transactions

View all successful transactions:

```
SELECT * FROM Payment
WHERE status = 'Success';
```

View all failed transactions:

```
SELECT * FROM Payment
WHERE status = 'Failed';
```

Update a failed transaction after retry:

```
UPDATE Payment
SET status = 'Success'
WHERE payment_id = 103;
```

## 4. Analyzing Payment Methods Used by Customers

Count of transactions per payment mode:

```
SELECT payment_mode, COUNT(*) AS total_transactions
FROM Payment
GROUP BY payment_mode
ORDER BY total_transactions DESC;
```

Most preferred payment mode by each customer:

```
SELECT customer_id, payment_mode, COUNT(*) AS uses
FROM Payment
WHERE status = 'Success'
GROUP BY customer_id, payment_mode
ORDER BY customer_id, uses DESC;
```

## 5. Generating Payment Transaction Reports

Daily transaction summary report:

```
SELECT payment_date,
       COUNT(*) AS total_transactions,
       SUM(CASE WHEN status='Success' THEN 1 ELSE 0 END) AS successful,
       SUM(CASE WHEN status='Failed' THEN 1 ELSE 0 END) AS failed,
       SUM(CASE WHEN status='Success' THEN amount ELSE 0 END) AS total_amount_collected
FROM Payment
GROUP BY payment_date
ORDER BY payment_date;
```

Overall success/failure ratio report:

```
SELECT status, COUNT(*) AS count,
       ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM Payment), 2) AS percentage
FROM Payment
GROUP BY status;
```

## Sample Payment Table Data

| payment_id | order_id | customer_id | mode        | date       | amount  | status  |
|------------|----------|-------------|-------------|------------|---------|---------|
| 1          | 101      | 1           | UPI         | 2026-08-01 | 2500.00 | Success |
| 2          | 102      | 2           | Credit Card | 2026-08-02 | 1500.00 | Success |
| 3          | 103      | 3           | Net Banking | 2026-08-03 | 3200.00 | Failed  |

| payment_id | order_id | customer_id | mode       | date       | amount  | status  |
|------------|----------|-------------|------------|------------|---------|---------|
| 4          | 104      | 1           | Cash       | 2026-08-04 | 800.00  | Success |
| 5          | 105      | 4           | UPI        | 2026-08-05 | 1200.00 | Failed  |
| 6          | 106      | 2           | Debit Card | 2026-08-06 | 4500.00 | Success |

Conclusion: The Payment table successfully models real-world transaction data, enabling tracking of payment mode, date, and status; separation of successful and failed transactions; analysis of customer payment preferences; and generation of summarized transaction reports.